

A MINI PROJECT
REPORT
ON
PAYROLL
MANAGEMENT SYSTEM

Table of Contents

ACKNOWLEDGEMENT.....	4
LIST OF FIGURES.....	5
CHAPTER-1 INTRODUCTION.....	6-10
OVERVIEW OF THE PROJECT.....	6
NEED FOR A PAYROLL MANAGEMENT SYSTEM	6-7
OBJECTIVES OF THE PROJECT.....	7
SCOPE OF THE PROJECT.....	8
SYSTEM FEATURES.....	8-9
MODULES OF THE SYSTEM.....	10
CHAPTER 2 LITERATURE REVIEW.....	11-19
REVIEW OF PREVIOUS WORK.....	11-14
RESEARCH GAPS IDENTIFIED.....	14-16
THEORITICAL BACKGROUD.....	17-19
CHAPTER-3 PROPOSED METHODOLOGY.....	20-28
SYSTEM DESIGN.....	20-22
SYSTEM ARCHITECTURE.....	23
FLOWCHARTS.....	24
ER DIAGRAM.....	25
DATA FLOW DIAGRAM.....	26
TECHNICAL APPROACH.....	27-28
CHAPTER-4 IMPLEMENTATION AND RESULTS.....	29-43
MODULE EXPLANATION AND CODING SCREENSHOTS.....	29-40
TEST RESULTS / OUTPUTS.....	41-43
CHAPTER-5 PROJECT CONCLUSION.....	44
CHAPTER-6 FUTURE SCOPE.....	45

LIST OF FIGURE

Figure No.	Description	Page No.
Figure 2.1	System Architecture Diagram	23
Figure 3.1	Main Menu Flowchart	24
Figure 3.2	Flowchart for Adding Employee	24
Figure 4.1	ER Diagram	25
Figure 5.1	DFD Level 0	26
Figure 5.2	DFD Level 1	26

CHAPTER 1 - INTRODUCTION

1.1 Overview of the Project

A Payroll Management System is a software application designed to automate, monitor, and manage employee salary information within an organization. Payroll processing involves several key functions, such as calculating basic salary, allowances, deductions, taxes, and generating salary slips. In manual systems, these tasks often consume time and lead to calculation errors, especially when the number of employees grows.

This project aims to implement an efficient Payroll Management System using Python and MySQL, where all employee salary-related data is securely stored in a database and processed using automated logic. The system provides a simple menu-driven program that enables users to add employees, modify data, generate payrolls, and view salary slips.

This project is a practical demonstration of database operations, Python programming, modular logic-building, SQL concepts, and real-world application development

1.2 Need for a Payroll Management System

Many small and medium-sized organizations face difficulties managing payroll manually because:

- Salary calculations become lengthy and complex
- Human errors occur frequently
- Modifying or retrieving records becomes difficult
- Paper-based or spreadsheet systems are not secure
- Storing long-term data is challenging

The computerized Payroll Management System helps overcome these limitations. It:

- Automates all salary calculations
- Stores employee records securely in SQL database
- Allows fast retrieval and modification
- Ensures data accuracy and consistency
- Provides a structured and organized method for payroll preparation

This makes payroll management efficient, error-free, and reliable.

1.3 Objectives of the Project

The main objectives of the Payroll Management System are as follows:

(3.1) Automation of Payroll Calculations

To automatically compute DA (Dearness Allowance), HRA (House Rent Allowance), gross salary, tax, and net salary based on employee position and basic salary.

(3.2) Centralized Employee Data Management

To maintain all employee information in a **single, secure MySQL database** for easy access and modification.

(3.3) User-Friendly Operation

To provide a simple interface where the user can perform all operations through menu-driven commands.

(3.4) Reduction of Manual Workload

To reduce time and effort required for maintaining payroll manually.

(3.5) Fast Retrieval and Reporting

To instantly generate payroll sheets and salary slips for internal and external verification.

1.4 Scope of the Project

The system has been designed to manage the salary-related operations of employees in an organization. The scope includes:

- Adding new employee records
- Displaying all employee details
- Searching for a particular employee
- Updating salary or job-related information
- Automatic computation of salary components
- Deleting employee information when required
- Providing complete payroll reports
- Generating detailed salary slips

The project is suitable for offices, institutes, shops, and small organizations, and can be easily extended into a larger HR management system.

1.5 System Features

The Payroll Management System includes the following core functionalities:

(1.5.1) Add Employee Record

Allows entry of new employee details such as name, post, and basic salary. The program automatically calculates DA, HRA, tax, gross salary, and net salary.

(1.5.2) Display All Employee Records

Shows the stored data of all employees in a structured tabular format.

(1.5.3) Display Particular Employee Record

Retrieves salary information of a specific employee using employee number.

(1.5.4) Delete Employee Record

Deletes the record of an employee when the user requests.

(1.5.5) Display Payroll

Generates a complete payroll list showing salary components for every employee.

(1.5.6) Increase an Employee's Salary

Increases basic salary and automatically recalculates other salary components.

(1.5.7) Display Salary Slip

Generates a structured salary slip showing all components including net salary.

(1.5.8) Modify Employee Record

Allows modification of name, post, or salary values with auto-recalculation.

(1.5.9) Exit Option

Ends the program smoothly.

1.6 Modules of the System

To simplify design and functioning, the system is divided into multiple modules:

(1.6.1) Database Creation Module

Creates database and employee table if not already present.

(1.6.2) Employee Entry Module

Accepts data from the user and inserts it into the database.

(1.6.3) Search and Display Module

Helps search and display employee details.

(1.6.4) Payroll Generation Module

Displays salary details for all employees collectively.

(1.6.5) Salary Slip Module

Generates a formal slip for individual employees.

(1.6.6) Update Module

Modifies existing records as required.

(1.6.7) Delete Module

Removes unwanted or old employee data.

(1.6.8) User Interface Module

Implements the menu-driven flow of the program.

CHAPTER 2 - LITERATURE REVIEW

2.1 REVIEW OF PREVIOUS WORK

A Payroll Management System is a well-studied problem domain in computer science, especially within the areas of database systems, management information systems (MIS), enterprise resource planning (ERP), and financial computing. Over the years, many researchers, organizations, and software developers have contributed to improving payroll accuracy, automation, and efficiency. This section provides a structured review of previous work in this domain.

2.1.1 Early Manual Payroll Systems

Earlier systems relied heavily on manual bookkeeping. Salary calculation, deduction entry, leave tracking, and tax computation were handwritten. These systems suffered from:

- High chances of clerical error
- Time-consuming calculations
- Difficulty in maintaining historical records
- Limited transparency for employees

Studies from early computerization phases highlighted that error rates in manual payroll were significantly high, sometimes reaching 5–10% per payroll cycle, especially in large organizations.

2.1.2 First-Generation Digital Payroll Systems (1980s–1990s)

With the rise of early computers, basic software tools (mostly COBOL-based) began automating payroll calculations. Research from this period emphasized:

- Use of structured programming
- Batch processing
- Storage using flat files
- Static salary rules incorporated directly into code

These systems improved accuracy but were still difficult to update. Any change in tax rules or allowances required rewriting significant code blocks.

2.1.3 Second-Generation Payroll Systems (2000–2010)

This era introduced more advanced systems using relational databases such as MySQL, SQL Server, and Oracle. Research focused on:

- Multi-table relational database design
- Data normalization to reduce redundancy
- Accounting integration
- GUI-based desktop payroll applications
- Role-based access and authentication

These systems demonstrated major improvements in scalability and security but lacked portability (mostly Windows-based desktop applications).

2.1.4 Web-Based and Cloud Payroll Systems (2010–Present)

In the modern era, payroll systems have transformed into cloud-based, SaaS-driven applications. Most research in this period focuses on:

- Web-based payroll accessibility
- Encrypted storage and security
- Automated tax rule updates
- Integration with attendance systems
- Real-time analytics and reporting
- Use of APIs for HR, finance, and attendance data integration

Modern systems, like those built using Python, Java, PHP, and cloud technologies, aim to reduce human interaction and improve accuracy even further.

2.1.5 Mobile-Integrated Payroll Systems (Recent)

With smartphones becoming universal, many studies highlight mobile-based payroll apps that allow employees to:

- View salary slips
- Apply for leaves
- Track attendance
- View tax deductions
- Update personal data

Such works showed improved employee satisfaction and reduced HR workload.

2.1.6 Machine Learning & AI-Based Payroll Systems (Emerging)

Recent academic and industrial work is exploring:

- Predictive salary modeling
- Fraud detection in payroll
- Automatic anomaly detection
- Chatbot-based HR support

Although highly advanced, these systems require huge datasets and advanced analytics techniques.

2.2 RESEARCH GAPS IDENTIFIED

Despite rich history and evolution, several gaps still exist that justify the development of simplified and academic-level payroll systems such as the one in this project.

2.2.1 Lack of Simple, Educational, and Modular Systems for Students

Many existing solutions are too complex and enterprise-oriented. There is a lack of simple, modular Payroll Management Systems suitable for:

- Learning database connectivity
- Understanding CRUD operations
- Practicing SQL queries
- Exploring procedural logic

This project fills this gap by offering a clean, understandable implementation using Python + MySQL.

2.2.2 Limited Documentation for Beginner-Level Implementations

Many research papers focus on enterprise-scale solutions, ignoring small-scale educational contexts. Clear explanations of:

- Allowance calculations
 - Tax automation
 - Salary modification procedures
 - Record integrity
- are often missing.

This project addresses the gap by providing a fully-documented and logically clear design.

2.2.3 Lack of Offline/Local Payroll Systems

Most modern payroll solutions are online or cloud-based. However, many small businesses or academic labs need:

- Offline, secure, local databases
- No dependency on internet
- Customizable salary rules

This project answers that need.

2.2.4 Need for Transparent Calculation Logic

Modern software hides internal calculation logic behind UI layers. Beginners rarely get to see actual calculation formulas.

This project explicitly demonstrates:

- DA, HRA calculation
- Gross salary computation
- Tax deductions
- Net salary formulation

Thus, it fills an important educational research gap.

2.2.5 Inadequate Employee-Focused Features in Small-Scale Systems

Most small-scale systems lack:

- Individual salary slips
- Record modification options
- Salary increment operations

This project includes these features.

2.3 THEORETICAL BACKGROUND

The Payroll Management System relies on several theoretical foundations from computer science and financial management. These theories form the base upon which the system is designed and implemented.

2.3.1 Database Management Theory

Payroll is a highly structured data domain requiring accuracy and consistency.

Theoretical foundations include:

2.1.1 Relational Database Model

Developed by E.F. Codd, this model emphasizes:

- Tables (relations)
- Rows (tuples)
- Columns (attributes)
- Keys (primary/foreign keys)

In this project, the emp table corresponds to a single relation holding all salary-related attributes.

2.1.2 Data Integrity Principles

To ensure valid and accurate data, principles include:

- *Entity integrity* (no duplicate employee numbers)
- *Domain integrity* (correct data types)
- *Referential integrity* (consistency across operations)

2.1.3 SQL and Query Processing

SQL concepts used include:

- DDL (CREATE DATABASE, CREATE TABLE)
- DML (INSERT, UPDATE, DELETE, SELECT)
- Constraints (PRIMARY KEY, NOT NULL)

These principles ensure reliable payroll operations.

2.3.2 Procedural Programming Theory

The project uses Python, following basic programming concepts such as:

- Conditionals (if-else)
- Loops (while)
- Functions and modularity
- Exception handling

These concepts help implement user-driven choices and automated computations.

2.3.3 Financial Computation Theory

Payroll systems are built on clear mathematical formulas. Important concepts include:

2.3.1 Basic Salary (Fixed Pay)

The core salary on which all allowances and deductions are calculated.

2.3.2 Allowances

- **DA (Dearness Allowance):** Compensates inflation.
- **HRA (House Rent Allowance):** Support for living expenses.

Both percentages vary by employee post.

2.3.3 Gross Salary

$\text{GrossSalary} = \text{Basic} + \text{DA} + \text{HRA}$

$\text{GrossSalary} = \text{Basic} + \text{DA} + \text{HRA}$

2.3.4 Tax Deduction

Tax is applied on basic salary in this project (simplified educational approach).

2.3.5 Net Salary

$\text{NetSalary} = \text{GrossSalary} - \text{Tax}$

2.3.4 Human–Computer Interaction (HCI) Theory

The menu-driven interface in the program uses standard HCI principles:

- Clarity
- Simple navigation
- Minimal cognitive load
- Helpful prompts
- Error messages

This ensures smooth user interaction.

2.5 Software Engineering Principles

2.5.1 Modularity

Each operation (add employee, delete employee, display payroll, etc.) acts as a separate functional module.

2.5.2 Maintainability

The code allows easy updates, such as modifying salary logic or adding new roles,

CHAPTER 3 - PROPOSED METHODOLOGY

3.1 System Design

3.1.1 System Requirements

✓ Software Requirements

- Python 3.x
- MySQL Database
- mysql.connector module

✓ Hardware Requirements

- Minimum 4GB RAM
- Dual-core Processor
- 500 MB storage

3.1.2 System Design Overview

The Payroll Management System in the program follows:

- **Input Layer**

User interacts through the console menu.

- **Processing Layer**

Python program performs:

- Salary calculations
- DA, HRA, TAX rules based on post
- Updating database

- **Database Layer**

MySQL table `emp` stores all employee details.

3.1.3 Salary Calculation Logic (based on your code)

Officer

- DA = 50% of Basic
- HRA = 35% of Basic
- Tax = 20% of Basic

Manager

- DA = 45% of Basic
- HRA = 30% of Basic
- Tax = 15% of Basic

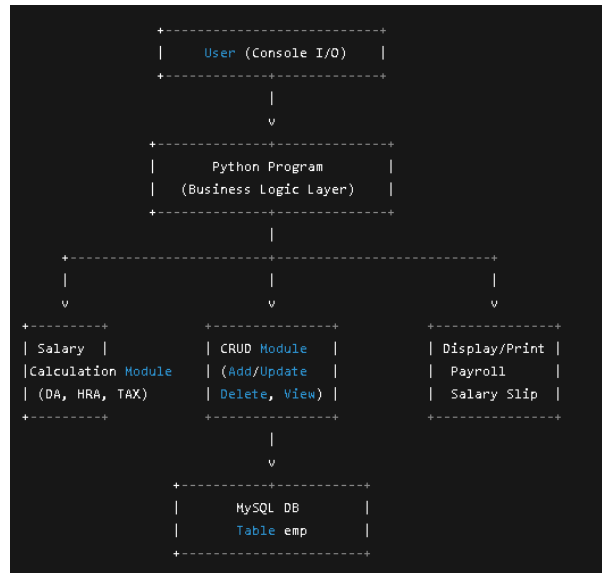
Other Posts

- DA = 40% of Basic
- HRA = 25% of Basic
- Tax = 10% of Basic

Gross Salary = Basic + DA + HRA

Net Salary = Gross – Tax

3.2 SYSTEM ARCHITECTURE (DIAGRAM + MODULES)



3.2.1 System Architecture Diagram

3.2.2 Architecture Explanation

The architecture is **three-tier**:

1. Presentation Layer

- Command Line Interface
- Menu-driven choices (1–9)

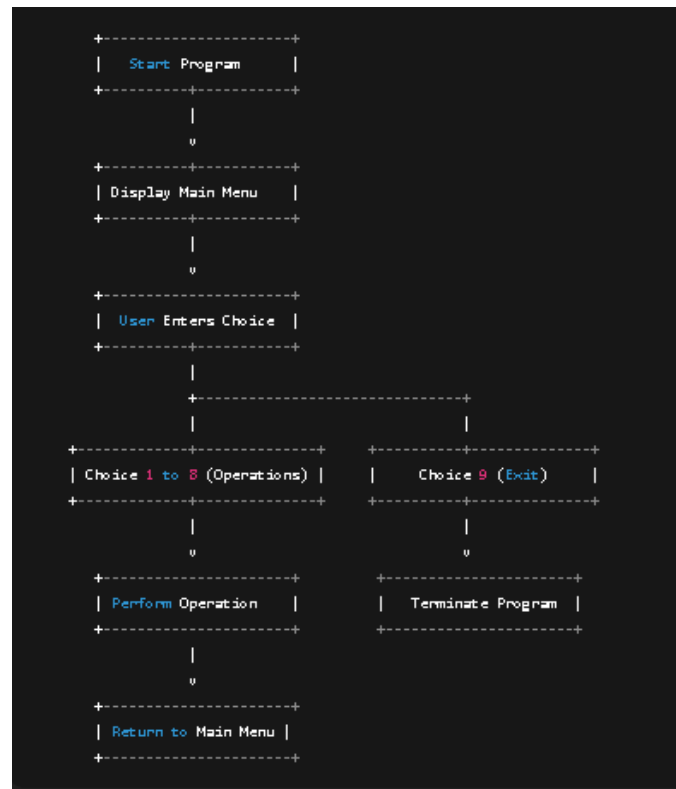
2. Application Logic Layer

- Written in Python
- Handles all calculations and database queries

3. Database Layer

- Stores persistent employee records in MySQL
- Table: emp(EmpNo, Name, Post, BasicSalary, DA, HRA, GrossSalary, Tax, NetSalary)

3.3 FLOWCHARTS

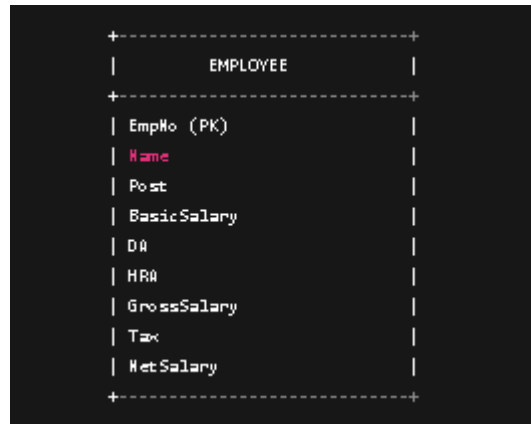


3.3.1 Main Menu Flowchart



3.3.2 Flowchart for Adding Employee

3.4 ER DIAGRAM + DATABASE DESIGN



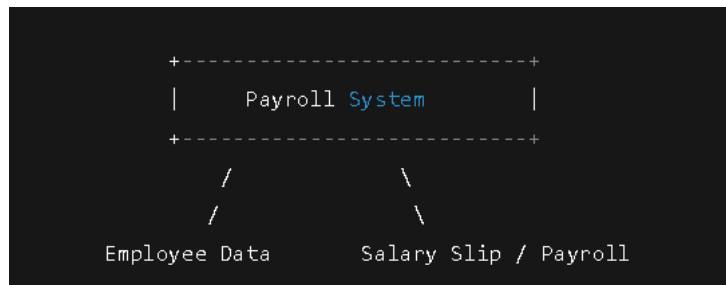
3.4.1 ER DIAGRAM

Entity: Employee

Primary Key: EmpNo

All attributes belong to the same entity, as the system is single-table based.

3.5 DATA FLOW DIAGRAMS (DFD)



3.5.1 DFD Level 0



3.5.2 DFD Level 1

3.6 TECHNICAL APPROACH

3.6.1 Technologies Used

Component	Technology
Programming Lang.	Python
Database Engine	MySQL
Connector	mysql.connector
Environment	Localhost
UI	Console-based

3.6.2 Salary Calculation Approach

The system dynamically computes salary components based on the "Post".

Example (from code)

```
if epost.upper()=='OFFICER':  
    eda=ebasic*0.5  
    ehra=ebasic*0.35  
    etax=ebasic*0.2
```

This ensures:

- Uniform salary structure
- Automatic recalculation on update
- No manual error

3.6.3 CRUD Operations

Add Employee

INSERT INTO emp VALUES (...)

View All / One Employee

SELECT * FROM emp

Update Employee

UPDATE emp SET ... WHERE EmpNo=%s

Delete Employee

DELETE FROM emp WHERE EmpNo=%s

CHAPTER 4 - IMPLEMENTATION AND RESULTS

Module Explanation and Coding Screenshots (Source Code)

4.1 Displaying Program Title

Code:

```
print("***90")
print("\t\t\t\tPAYROLL MANAGEMENT SYSTEM")
print("***90")
```

4.2 Database Initialization Module

This module runs at the beginning of the program.

Its functions are:

- Connect to MySQL
- Create database `payroll` if it does not exist
- Select database
- Create table `emp` if it does not exist

This ensures the system always starts with the required database environment and maintains persistent storage of employee data.

Code:

```
#creating database
import mysql.connector as con
c=con.connect(host="localhost", user="root", passwd="123456")
cur=c.cursor()
query="Create database if not exists payroll"
cur.execute(query)
cur.execute("Use payroll")
```

4.3 Table Creation Module

This simply creates a table named emp and writes a query in a string then executes it in sql by using the execute function.

Code:

```
#creating table
sql="Create table if not exists emp(EmpNo int primary key,\
Name varchar(20) not null,Post varchar\
(15),BasicSalary int,DA float,HRA float, GrossSalary float,\
Tax float,NetSalary float)"
cur.execute(sql)
c.commit()
```

4. Menu Module

It displays a whole menu to perform the following action and takes a choice from the user and act accordingly:

- 1: To Add Employee Records
- 2: To Display Records of All the Employees
- 3: To Display Record of a Particular Employee
- 4: To Delete Record of Particular Employee
- 5: To Display Payroll
- 6: To Increase an Employee's Salary
- 7: To Display Salary Slip of a Particular Employee
- 8: To Modify a Record
- 9: To Exit

Code:

```
while True:
    print()
    print()
    print(""*90)
    print("\t\t\t\t\tMAIN MENU")
    print(""*90)
    print()
    print("Press any number from 1 to 10 to execute any of the desired procedures give")
    print()
    print("\t1: To Add Employee Records")
    print("\t2: To Display Records of All the Employees")
    print("\t3: To Display Record of a Particular Employee")
    print("\t4: To Delete Record of Particular Employee")
    print("\t5: To Display Payroll")
    print("\t6: To Increase an Employee's Salary")
    print("\t7: To Display Salary Slip of a Particular Employee")
    print("\t8: To Modify a Record")
    print("\t9: To Exit")
    print()
    ch=int(input('Enter Your Choice:'))
```

5. Add Employee Module (Choice 1)

This module collects employee details such as:

- Employee Number
- Name
- Post
- Basic Salary

Based on the post (Officer / Manager / Others), the system automatically calculates:

- Dearness Allowance (DA)
- House Rent Allowance (HRA)
- Tax
- Gross Salary
- Net Salary

It inserts the complete record into the MySQL table.

This ensures standardized salary structure and reduces user errors by automating salary computation.

```
#Procedure to Add Employees
if ch==1:
    print("Enter Employee Information.....")
    eno=int(input("Enter EmpNo.:"))
    ename=input("Enter Employee's Name:")
    epost=input("Enter Employee's Post:")
    ebasic=float(input("Enter Employee's Basic Salary:"))
    if epost.upper()=='OFFICER':
        eda=ebasic*0.5
        ehra=ebasic*0.35
        etax=ebasic*0.2
    elif epost.lower()=='manager':
        eda=ebasic*0.45
        ehra=ebasic*0.30
        etax=ebasic*0.15
    else:
        eda=ebasic*0.40
        ehra=ebasic*0.25
        etax=ebasic*0.1
    egross=ebasic+eda+ehra
    enet=egross-etax
    S=(eno,ename,epost,ebasic,eda,ehra,egross,etax,enet)
    S="insert into emp values (%s,%s,%s,%s,%s,%s,%s,%s,%s)"
    cur.execute(S,S)
    c.commit()
    print("RECORDS ADDED SUCCESSFULLY !!!!!!!")
    print()
```

Code:

6. Display All Employee Records Module (Choice 2)

This module retrieves and displays all rows from the employee table.

It uses:

```
SELECT * FROM emp
```

The entire set of employee records is shown in tuple form.

This helps administrators view complete database contents at any time.

Code:

```
#Procedure to Display Records of All Employees
elif ch==2:
    S="Select * from emp"
    cur.execute(S)
    data=cur.fetchall()
    for i in data:
        print(i)
    print()
```

7. Display One Employee Module (Choice 3)

This module retrieves a single employee record by EmpNo.

If the employee exists, the record is displayed; otherwise, a "Record Not Found" message appears.

This ensures quick lookup of employee salary information or verification for HR purposes.

Code:

```
#Procedure to Display Records of A Particular Employee
elif ch==3:
    empno=int(input("Enter Emp No.:"))
    S="Select * from emp where EmpNo=%s"%(empno)
    cur.execute(S)
    data=cur.fetchone()
    if data==None:
        print("Record not found")
        print()
        print("***90)
    else:
        print(data)
        print()
```

8. Delete Employee Record Module (Choice 4)

This module deletes an employee's data from the database using:

```
DELETE FROM emp WHERE EmpNo = ...
```

It removes outdated or incorrect entries and maintains clean database records.

Confirmation is shown after deletion.

Code:

```
#Procedure to Delete Record of A Particular Employee
elif ch==4:
    try:
        empno=int(input("Enter Emp No.:"))
        S="Delete from emp where EmpNo=%s"%(empno)
        cur.execute(S)
        c.commit()
        print("Record Deleted Successfully !!!!!!")
        c.commit()
        print()
    except:
        print("Something Went Wrong !!!!!!")
        print()
```

9. Display Payroll Module (Choice 5)

This module prints a formatted table of all employee salary details.

Displayed fields:

- EmpNo
- Name
- Post
- Basic Salary
- DA
- HRA
- Gross Salary
- Tax
- Net Salary

This acts as a payroll register used by HR departments and accountants.

It simulates real-world payroll reports.

Code:

```
#Procedure To Display Payroll
elif ch==5:
    S="Select * from emp"
    cur.execute(S)
    data=cur.fetchall()
    print("***100")
    print("PAYROLL".center(90))
    print("***100")
    print("| EmpNo | Name | Post | BasicSalary | DA | \
HRA | GrossSalary | Tax | NetSalary |")
    for i in data:
        print("| ",i[0]," | ",i[1]," |",i[2]," | ",i[3]," | ",i[4]," |",i[5],
              ,i[6]," |",i[7]," | ",i[8]," |")
    print()
```

10. Increase Salary Module (Choice 6)

This module:

1. Takes an amount to increase.
2. Adds the amount to the employee's Basic Salary.
3. Recalculates DA, HRA, Gross, Tax, Net Salary based on the new Basic Salary.
4. Updates the database record.

This feature is essential for promotions, annual increments, or bonuses.

It dynamically recomputes all dependent salary components to avoid manual calculation errors.

Code:

```
#Procedure to Increase The Salary of An Employee
elif ch==6:
    amt=int(input("Enter amount of salary to be increased:"))
    eno=int(input("Enter Emp No.:"))
    cur.execute("Select * from emp")
    d=cur.fetchall()
    for i in d:
        if i[0]==eno:
            cur.execute("Select * from emp where EmpNo=%s"%(eno))
            data=cur.fetchone()
            t=data[3]+amt
            if data[2].upper()=='OFFICER':
                da=t*0.5
                hra=t*0.35
                tax=t*0.2
            elif data[2].lower()=='manager':
                da=t*0.45
                hra=t*0.30
                tax=t*0.15
            else:
                da=t*0.40
                hra=t*0.25
                tax=t*0.1
            g=t+da+hra
            n=g-tax
            cur.execute("Update emp set BasicSalary=%s,DA=%s,HRA=%s,GrossSalary=
Tax=%s,NetSalary=%s where EmpNo='%s'"%(t,da,hra,g,tax,n,eno))
            c.commit()
            print("Updated Successfully !!!!!!!")
            print()
            n=1
            break
    if n!=1:
        print("Record Not Found !!!!!")
        print()
```

11. Salary Slip Generation Module (Choice 7)

This module creates a formatted salary slip for a single employee.

The slip includes:

- Name
- Basic Salary
- DA
- HRA
- Tax
- Net Salary

It resembles a real payslip used in organizations and can be used for documentation, reimbursement, or auditing.

Code:

```
#Procedure to Display Salary Slip of A Particular Employee
elif ch==7:
    eno=int(input("Enter Emp No. to display the Salary Slip:"))
    print()
    cur.execute("Select * from emp")
    d=cur.fetchall()
    for i in d:
        if i[0]==eno:
            print("***90)
            print("Salary Slip".center(90))
            print("***90)
            print()
            cur.execute("Select Name,BasicSalary,DA,HRA,Tax,NetSalary from emp where EmpNo="+str(eno))
            d=cur.fetchone()
            print("-----")
            print("|      Name      :",d[0]," |")
            print("|Basic Salary:",d[1]," |")
            print("|      DA       :",d[2]," |")
            print("|      HRA      :",d[3]," |")
            print("|      Tax       :",d[4]," |")
            print("| Net Salary  :",d[5]," |")
            print("-----")
            print()
            n=1
            break
        else:
            n=2
    if n==2:
        print("Record Not Found !!!!!")
        print()
```

12. Modify Record Module (Choice 8)

This module allows partial or complete editing of an employee's data.

The user can change:

- Name
- Post
- Basic Salary

Whenever Post or Basic Salary is modified, the system automatically recalculates salary components.

This ensures updated and accurate records even after structural changes in salary.

Code:

(1)

```
#Procedure to Modify A Record
elif ch==8:
    eno=int(input("Enter Emp No. Whose Record Is To Be Modified:"))
    cur.execute("Select * from emp")
    data=cur.fetchall()
    for i in data:
        if i[0]==eno:
            cur.execute("Select * from emp where EmpNo=%s"%(eno))
            d=cur.fetchone()
            ename=d[1]
            epost=d[2]
            ebasic=d[3]
            eda=d[4]
            ehra=d[5]
            eg=d[6]
            etax=d[7]
            en=d[8]
            print("Employee Id      :",d[0])
            print("Name              :",d[1])
            print("Post               :",d[2])
            print("Basic Salary        :",d[3])
            print("Dearing Allowance   :",d[4])
            print("House Rent Allowance:",d[5])
            print("Gross Salary        :",d[6])
            print("Tax                 :",d[7])
            print("Net Salary          :",d[8])
            print("-"*45)
            print("Enter Value To Modify Below or Just Press Enter For No Change...")
            x=input("Enter Name:")
```

(2)

```
if len(x)>0:
    ename=x
x=input("Enter Post:")
if len(x)>0:
    epost=x
    if epost.upper()=='OFFICER':
        eda=ebasic*0.5
        ehra=ebasic*0.35
        etax=ebasic*0.2
    elif epost.lower()=='manager':
        eda=ebasic*0.45
        ehra=ebasic*0.30
        etax=ebasic*0.15
    else:
        eda=ebasic*0.40
        ehra=ebasic*0.25
        etax=ebasic*0.1
    eg=ebasic+eda+ehra
    en=eg-etax
x=input("Enter Basic Salary:")
if len(x)>0:
    ebasic=float(x)
    if epost.upper()=='OFFICER':
        eda=ebasic*0.5
        ehra=ebasic*0.35
        etax=ebasic*0.2
    elif epost.lower()=='manager':
        eda=ebasic*0.45
        ehra=ebasic*0.30
        etax=ebasic*0.15
    else:
        eda=ebasic*0.40
        ehra=ebasic*0.25
        etax=ebasic*0.1
    eg=ebasic+eda+ehra
    en=eg-etax
pes=(ename,epost,ebasic,eda,ehra,eg,etax,en,eno)
S="Update emp set Name='%s',Post='%s',BasicSalary=%s,DA=%s,HRA=%s,GrossSalary=%s,\
Tax=%s,NetSalary=%s where EmpNo=%s"%(pes)
.....
```

(3)

```
print("Records Modified Sucessfully !!!!!")
cur.execute(S)
c.commit()
n=1
break
if n!=1:
    print("Record Not Found !!!!!")
```

13. Exit Module (Choice 9)

Ends the loop and terminates program execution gracefully.

It ensures the system closes safely without incomplete transactions.

Code:

```
#Procedure To Exit
elif ch==9:
    print()
    print("-X"*50)
    break

else:
    print("Wrong Input !!!!!")
```

Test Results / Outputs

Menu Output:

```
*****
PAYROLL MANAGEMENT SYSTEM
*****

*****
MAIN MENU
*****

Press any number from 1 to 10 to execute any of the desired procedures given below

1: To Add Employee Records
2: To Display Records of All the Employees
3: To Display Record of a Particular Employee
4: To Delete Record of Particular Employee
5: To Display Payroll
6: To Increase an Employee's Salary
7: To Display Salary Slip of a Particular Employee
8: To Modify a Record
9: To Exit

Enter Your Choice:
```

Choice 1:

```
Enter Your Choice:1
Enter Employee Information.....
Enter EmpNo.:1
Enter Employee's Name:Mohammed Hamza Saeed
Enter Employee's Post:Officer
Enter Employee's Basic Salary:75000
RECORDS ADDED SUCCESSFULLY !!!!!!!
```

Choice 2:

```
Enter Your Choice:2
(1, 'Mohammed Hamza Saeed', 'Officer', 75000, 37500.0, 26250.0, 138750.0, 15000.0,
(2, 'Mohd Waris Ansari', 'Manager', 45000, 20250.0, 13500.0, 78750.0, 6750.0, 7200
(3, 'Rahul Gupta', 'Accountant', 25000, 10000.0, 6250.0, 41250.0, 2500.0, 38750.0)
(4, 'Aditya Srivastava', 'clerk', 15000, 6000.0, 3750.0, 24750.0, 1500.0, 23250.0)
(5, 'Rehan Khan', 'Assistant', 18000, 7200.0, 4500.0, 29700.0, 1800.0, 27900.0)
```

Choice 3:

```
Enter Your Choice:3
Enter Emp No.:1
(1, 'Mohammed Hamza Saeed', 'Officer', 75000, 37500.0, 26250.0, 138750.0, 15000.0,
```

Choice 4:

```
Enter Your Choice:4
Enter Emp No.:1
Record Deleted Successfully !!!!!
```

Choice 5:

```
Enter Your Choice:5
*****
                        PAYROLL
*****
| EmpNo | Name   | Post   | BasicSalary | DA   | HRA   | GrossSalary | Tax   | NetSalary |
| 1     | Hamza  | Officer | 75000       | 37500.0 | 26250.0 | 138750.0   | 15000.0 | 123750.0 |
| 2     | Waris  | Manager | 45000       | 20250.0 | 13500.0 | 78750.0    | 6750.0  | 72000.0  |
| 3     | Rehan  | Clerk  | 15000       | 6000.0  | 3750.0  | 24750.0    | 1500.0  | 23250.0  |
| 4     | Arjun  | Assistant | 20000      | 8000.0  | 5000.0  | 33000.0    | 2000.0  | 31000.0  |
| 5     | Deepika | Accountant | 21000      | 8400.0  | 5250.0  | 34650.0    | 2100.0  | 32550.0  |
```

Choice 6:

```
Enter Your Choice:6
Enter amount of salary to be increased:5000
Enter Emp No.:5
Updated Successfully !!!!!!!
```

Choice 7:

```

Enter Your Choice:7
Enter Emp No. to display the Salary Slip:1

*****
                        Salary Slip
*****

-----
|      Name       : Hamza      |
|Basic Salary:    : 75000      |
|      DA         : 37500.0    |
|      HRA        : 26250.0    |
|      Tax        : 15000.0    |
| Net Salary     : 123750.0    |

```

Choice 8:

```

Enter Your Choice:8
Enter Emp No. Whose Record Is To Be Modified:3
Employee Id      : 3
Name             : Rehan
Post             : Clerk
Basic Salary     : 15000
Dearing Allowance : 6000.0
House Rent Allowance: 3750.0
Gross Salary     : 24750.0
Tax              : 1500.0
Net Salary       : 23250.0
-----
Enter Value To Modify Below or Just Press Enter For No Change...
Enter Name:Post
Enter Post:Manager
Enter Basic Salary:35000
Records Modified Sucessfully !!!!!

```

Choice 9:

[illegible]

CHAPTER 5 - CONCLUSION

The “Payroll Management System” project successfully demonstrates how database programming and Python can be used together to automate real-life organizational tasks. The primary outcome of this project is the development of a functional, menu-driven software that stores, processes, and manages employee payroll data accurately and efficiently. The system integrates MySQL as the backend and Python as the frontend, enabling smooth data entry, calculation, retrieval, and modification.

Throughout the development of this project, several core objectives were achieved. The system successfully performs essential payroll functions such as adding employee records, displaying all or specific data, calculating gross and net salaries, generating salary slips, modifying and updating salary details, and deleting records. The logic for allowances (DA, HRA) and tax calculation based on employee posts (Officer, Manager, Others) ensures that payroll computation is realistic and adaptable for organizational use.

This project helped strengthen understanding of database connectivity using `mysql.connector`, SQL queries, CRUD operations, conditional statements, loops, and modular coding techniques. It also enhanced practical knowledge of how automated systems reduce manual errors and improve operational efficiency in HR and finance departments.

Overall, the project meets the expected learning outcomes of the AKTU B.Tech second-year curriculum by demonstrating analytical thinking, programming logic, file and database handling, and application development skills. The successful implementation of this system shows how software solutions can simplify organizational processes and highlights the importance of automation in modern workplaces.

CHAPTER 6 – FUTURE SCOPE

Although the current version of the Payroll Management System performs all essential payroll operations, there is significant scope for enhancement to make it more scalable, secure, and user-friendly in real-world usage. One major improvement would be to convert the command-line interface into a graphical user interface (GUI) using Tkinter, PyQt, or a web-based frontend. This would make the system accessible to non-technical users. Additionally, introducing login authentication and role-based access control (Admin, HR, Accountant) would improve security and prevent unauthorized modifications.

The system can also be expanded by integrating advanced payroll functions such as automatic tax calculation based on government rules, generating monthly reports, exporting data to Excel/PDF, and tracking attendance or leave records. Cloud integration with platforms like Firebase or AWS could enable companies to manage payroll from multiple branches in real time.

In future iterations, Machine Learning techniques can be added to predict salary increments, analyze employee performance trends, or detect payroll anomalies. Linking the software with biometric devices or attendance systems would help create a complete HR management suite. Adding REST APIs would allow connecting the payroll system to other applications such as ERP or accounting software.

In summary, the current system serves as a strong foundation, and with additional features like GUI development, automation, advanced reporting, cloud connectivity, multi-user authentication, and integration with other systems, it can evolve into a professional-grade payroll management solution suitable for large organizations.